

OpenZeppelin Sui AMM Audit



May 21, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Future Speculations	7
Medium Severity	8
M-01 Admin-Posted Quotes Can Be Overridden by Permissionless Refresh	8
M-02 Forced Re-Quoting via refresh_quotes_permissionless Causes FIFO Priority Loss on DeepBook	9
Low Severity	10
L-01 Quote Refresh Reverts on Non-Whitelisted DeepBook Pools	10
L-02 Quote Refresh Can Cross the Book and Trade as Taker	11
L-03 Expired Orders Stall Permissionless Quote Refresh	11
L-04 Orders Remain Active When Oracle Data Becomes Stale	12
Notes & Additional Information	13
N-01 Incorrect Documentation	13
N-02 Unused Functions	14
N-03 Code Style Inconsistencies	15
N-04 Arithmetic Edge Cases in Quote-Refresh Path Can Cause Aborts and Stale Orders	15
N-05 Spelling and Grammar Errors in Comments and Documentation Affect Readability	17
N-06 Live Order Ladder Quotes at Stale Parameters After update_config	18
N-07 Market Creation Allows Identical Pyth Feeds for Base and Quote	18
Conclusion	20
Appendix	21
Issue Classification	21

Summary

Type	Library	Total Issues	13 (10 resolved, 3 partially resolved)
Timeline	From 2026-04-28 To 2026-05-08	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	2 (1 resolved, 1 partially resolved)
		Low Severity Issues	4 (3 resolved, 1 partially resolved)
		Notes & Additional Information	7 (6 resolved, 1 partially resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [OpenZeppelin/openzeppelin-sui-amm](#) repository at commit [ab34cc9](#).

In scope were the following files:

```
packages
├── dapp
│   └── contracts
│       └── prop-amm
│           └── sources
│               ├── config.move
│               ├── events.move
│               ├── executor.move
│               ├── info.move
│               └── market.move
```

Update: The fixes for the findings highlighted in this report have all been merged at commit [827f744](#).

System Overview

The OpenZeppelin Sui market maker implements a proprietary automated market maker for the DeepBook v3 spot exchange on Sui. The package allows an operator to manage a single algorithmic market-making account that posts a four-order ladder (two bids and two asks) around an oracle mid price, with the goal of providing tighter quotes than a constant product or concentrated liquidity pool.

The package is composed of five modules that share a single owned object, the `Executor`. The `market` module encapsulates the asset pair identity, including the bound DeepBook pool, cached per-side decimals, and the Pyth price feed identifiers. The `config` module holds the AMM configuration values that drive quoting behaviour, including the inner spread, the volatility-driven outer spread, the inventory skew coefficient, the share of each side allocated to the outer order, and the oracle freshness limits. The `executor` module owns the central `Executor` object together with its embedded DeepBook `BalanceManager` and three DeepBook capabilities (`TradeCap`, `DepositCap`, `WithdrawCap`). The `info` module records the cumulative deposit, withdrawal, and post-settlement balance counters that off-chain consumers rely on. The `events` module emits the typed events that off-chain bots and indexers consume.

The trading lifecycle is gated by an `AdminCap` minted at construction. An operator creates the executor in a paused state, unpauses it, deposits base and quote balances, and then drives refreshes. The package exposes two refresh entrypoints.

`refresh_quotes_permissionless` is callable by any sender, reads the base and quote Pyth `PriceInfoObject` values, derives the DeepBook mid and the propagated confidence ratio, cancels existing orders, settles fills, and lays a new ladder. `refresh_quotes` is gated by the matching `AdminCap` and accepts a caller-supplied mid price and confidence ratio rather than reading Pyth, enabling off-chain market-data quoting. Both paths share the same cancel, settle, and ladder placement logic. Withdrawals require pausing first so that DeepBook settles all open orders into the `BalanceManager` before funds leave the executor.

Order placement targets DeepBook directly. The executor allocates the full per-side balance across the inner and outer orders according to `outer_balance_bps`, computes per-side prices from the `reservation mid` (the oracle mid shifted toward the rebalancing side per `inventory_skew_bps`), and submits each order with `pay_with_deep` set to `false`. The

package is bound to a single pool at creation time, and switching pools requires creating a new executor.

Security Model and Trust Assumptions

Admin (AdminCap holder): can unpause and pause trading, replace the AMM configuration through `update_config`, deposit and withdraw base or quote balances, drive the admin refresh path, and access the full set of cap-gated entry points. The admin is trusted to configure spreads, oracle freshness limits, and the inventory skew within values that produce well-formed orders, to keep the matching DeepBook pool consistent with the configured market metadata, and to monitor inventory and operational state. Loss of the `AdminCap` permanently locks the executor out of pausing, withdrawals, configuration changes, and the admin refresh path, so the cap should be held by an account with operational continuity guarantees.

Permissionless refresh endpoint: any sender that holds a reference to the executor and the relevant Pyth `PriceInfoObject` values can invoke `refresh_quotes_permissionless` when the executor is shared. The package does not authenticate this caller. The protocol relies on the freshness guard, which compares the cached publish timestamps with the incoming Pyth ones, and on the `max_conf_ratio_bps` confidence cap to block clearly invalid refreshes.

Pyth oracle: trusted as the on-chain source of base and quote USD prices for the permissionless path. The package validates that the supplied `PriceInfoObject` matches the feed identifier configured at creation, and reads each feed with `get_price_no_older_than` against the configured `max_price_age_secs`. Confidence-to-price ratios above `max_conf_ratio_bps` are rejected per leg, and the propagated combined confidence ratio drives the outer order's volatility buffer. Oracle integrity, including liveness, accuracy, and resistance to short-term manipulation of the published price or confidence interval, is assumed.

Beyond per-feed age and confidence bounds, the package implicitly trusts that the on-chain `PriceInfoObject` cache reflects the latest available Pyth publish for each feed at refresh time, and that the base and quote feeds advance approximately in lockstep. The freshness guard does not require the cached publish time to be the most recent one available off-chain,

nor does it constrain the skew between the base and quote publish timestamps. Both legs are validated independently against `max_price_age_secs`. As a consequence, the protocol relies on the off-chain Pyth update flow (and on callers acting in good faith when they elect which Pyth update to push on-chain ahead of `refresh_quotes_permissionless`) to keep both legs current and synchronized. A caller selecting an intermediate-but-not-latest cached publish, or refreshing immediately after only one leg advanced, can drive quoting against stale or time-skewed prices within the freshness window.

The package additionally assumes that both Pyth feeds publish strictly negative price exponents. The `deepbook_usd_price` helper aborts with `EPythExponentNonNegative` when the published exponent is zero or positive, so a feed update with a non-negative exponent would block oracle reads and prevent quote refreshes for the affected market.

DeepBook pool: trusted as the matching venue. The executor binds to a single pool by `pool_id` at creation and validates pool identity on every refresh.

Future Speculations

The configured `max_price_age_secs` applies to both the base and quote price legs through a single value. While Sui Pyth feeds in current production all share a comparable heartbeat, future feeds may publish at different cadences, and the present configuration cannot accommodate divergent freshness windows for the two legs without redeploying or extending the package.

The package does not implement an in-protocol versioning scheme. The `Executor`, `AdminCap`, and embedded `Market`, `AMMConfig`, and `Info` carry no version field, and entry points perform no version assertion. Future upgrades that require storage migration therefore depend on Sui's package upgrade mechanism and on external operational tooling rather than on in-protocol checks. An upgraded package cannot reject calls into stale `Executor` instances without an explicit migration path.

Medium Severity

M-01 Admin-Posted Quotes Can Be Overridden by Permissionless Refresh

The executor exposes two quote refresh entrypoints. `refresh_quotes_permissionless` is callable by any address and derives the mid price from the Pyth feeds, while `refresh_quotes` is gated by the `AdminCap` and accepts a caller-supplied `mid_price`. The [contract README](#) documents the admin path as the option to use "when quoting off off-chain market data (e.g. a CEX feed) is preferable" to Pyth. Both paths share the same precondition: the executor must be in the `active` state. There is no per-path enable flag in `AMMConfig`, and `refresh_quotes` itself requires `active = true`, so pausing the executor is not a workaround because it disables the admin path as well.

After the admin places quotes around an off-chain mid via the admin path, the cached Pyth publish timestamps are not advanced. The cache continues to reflect only the latest Pyth read, not the admin's most recent decision. The next Pyth feed update (which any address can pass into the permissionless path) therefore passes the [freshness check](#) and proceeds into `refresh_quotes_inner` with the Pyth-derived mid, cancelling the admin's resting orders and re-placing the four-order ladder at the Pyth price. The admin cannot keep off-chain-derived quotes posted while the executor is active. A third party can replace them within a single block of any Pyth update, and Pyth feed updates land frequently for active pairs. When the admin chose the off-chain feed because Pyth is unsuitable for the pair, for example because the on-chain confidence interval is wide or the Pyth mid drifts from the venue used to compute `mid_price`, the override pins the executor's quotes at the Pyth mid that was explicitly rejected and exposes them to fills at that price.

Consider propagating the admin's freshness into the cached Pyth publish times by calling `set_base_price_publish_time` and its quote-side equivalent with `clock.timestamp_ms() / 1000` at the end of `refresh_quotes`. The existing skip-stale guard then suppresses subsequent permissionless calls until Pyth publishes a strictly later timestamp. For indefinite lockout, additionally gate `refresh_quotes_permissionless` on a `permissionless_enabled: bool` field in `AMMConfig`.

Update: Partially Resolved in [pull request #147](#). The OpenZeppelin development team stated:

Timestamp update added inside admin refresh. Refresh quote permissionless configuration toggle omitted, since executor's owner has an option not to share executor or create additional not shared executor (and quickly move liquidity to it)

M-02 Forced Re-Quoting via `refresh_quotes_permissionless` Causes FIFO Priority Loss on DeepBook

The `refresh_quotes_permissionless` entrypoint is callable by any address whenever the executor is active, and the trade proof is [derived from the capability stored on the Executor](#). The only filter against unnecessary re-quoting is the [skip-stale guard](#), which short-circuits the call only when the executor has open orders and neither Pyth feed has advanced.

DeepBook V3 enforces strict FIFO priority per price level, with order IDs encoded as `(price << 64) + sequence` by `encode_order_id` and the sequence advanced inside `get_order_id` on every placement. Every refresh cancels the resting ladder and re-posts with fresh sequence numbers, placing each order at the tail of its FIFO queue. Because Pyth feeds advance every few seconds and can be pushed by anyone at low gas cost, a bot can call `refresh_quotes_permissionless` on every advance, forcing the executor to re-queue behind any maker that arrived since the previous refresh. The cost to the maker is FIFO priority loss bounded by the spread that would otherwise have been earned, and scales with the placement rate of competing makers, so actively-quoted pools are the most exposed.

Consider gating the permissionless path on a configurable per-call cooldown, for example, by tracking a `last_refresh_ms: u64` field on the `Executor` and asserting `clock.timestamp_ms() - self.last_refresh_ms >= self.config.permissionless_cooldown_ms()`, with the admin path bypassing the gate. Alternatively, comparing the newly-derived Pyth mid against the previously-quoted mid and skipping the refresh when the move is below a configurable fraction of `base_spread` preserves the freshness benefit while suppressing redundant cancel-and-replace cycles.

Update: Resolved in [pull request #146](#).

Low Severity

L-01 Quote Refresh Reverts on Non-Whitelisted DeepBook Pools

The `Executor` rebuilds its bid and ask ladder by invoking `refresh_quotes_inner`, which is reached from both the admin `refresh_quotes` endpoint and the permissionless `refresh_quotes_permissionless` endpoint. Inside that function, the post-settlement `BalanceManager` balances are read and split across two orders per side. The outer bid quantity is taken as a configured fraction of the quote balance and the `inner bid quantity is set to quote_balance - bid_outer_quantity_quote`, with the same partition applied to the `base balance on the ask side`. Each of the four resulting candidates is then forwarded to DeepBook's `pool.place_limit_order` through `try_place_limit_orders`, with `pay_with_deep = false` hardcoded.

When a DeepBook pool is created with `whitelisted_pool = true`, both the maker and taker fees are zero, so the no-DEEP setting is harmless. However, on any non-whitelisted pool the default volatile-pair schedule applies a 5 bps maker fee combined with the 1.25x non-DEEP penalty multiplier, for an effective 6.25 bps charge denominated in the input asset of the order. Because the executor allocates 100% of each side's balance across its two orders and reserves nothing for that fee, DeepBook's internal `withdraw_with_proof` finds the `BalanceManager` short by exactly the fee amount the moment the second order on a given side is placed. The call aborts with `EBalanceManagerBalanceTooLow` and the entire `refresh_quotes_inner` transaction reverts. The first refresh against any realistic mainnet pool fails, leaving the executor unable to quote at all and rendering the protocol's core market-making feature unusable. No funds are at risk, since the abort happens before any order rests on the book. The existing test suite does not surface the issue because every test pool is constructed with `whitelisted_pool = true`, which zeroes both fees.

Consider either explicitly documenting that only whitelisted DeepBook pools are supported by the executor, or adding a check during executor creation that asserts `pool.whitelisted()` returns `true`, so that incompatible pools are rejected up front instead of causing a failure at the first quote refresh.

Update: Resolved in [pull request #139](#).

L-02 Quote Refresh Can Cross the Book and Trade as Taker

In the `refresh_quotes_inner_function`, the executor cancels its resting orders and submits four new orders around a reservation mid derived from the oracle price and current inventory. Each order is placed with `order_type = constants::no_restriction()`, instructing DeepBook to fill any portion that crosses the book at submission, with only the unfilled remainder being rested.

If the DeepBook order book state changes between the keeper reading Pyth and the transaction being included, or if inventory skew or realized volatility exceeds the configured spreads, an order intended by the executor to be passive may cross with existing orders and execute as a taker. In this case, the pool acquires inventory at a price the executor would not have chosen on a fresh oracle reading. The existing `cancel_taker` self-matching guard does not prevent this, as it only blocks crosses against the executor's own resting orders.

Because `refresh_quotes_permissionless` is callable by anyone and the next reservation mid is deterministic from on-chain state, this behavior can be exploited by an attacker who rests a limit order just inside the executor's anticipated inner quote, then invokes a permissionless refresh and is filled at a favorable price while the AMM trades as a taker.

Consider placing the orders with `constants::post_only()` instead, which aborts with `EPOSTOrderCrossesOrderbook` if the order would execute against the book. This abort would roll back the cancel-and-replace performed at the start of `refresh_quotes`, preserving the previous quotes until the next refresh, which will use a new oracle reading.

Update: Resolved in [pull request #138](#).

L-03 Expired Orders Stall Permissionless Quote Refresh

The `skip-stale_guard` inside `refresh_quotes_permissionless` short-circuits the call when the executor has open orders and neither Pyth feed has advanced relative to the cached publish times. The `has_open_orders` helper checks `open_orders().is_empty()` on the executor's DeepBook account. DeepBook's `Account.open_orders` is mutated only on placement, on cancellation, and inside `process_fill`, which removes a maker order ID only when a matching attempt produces a fill marked completed or expired. Expired orders are therefore removed lazily; without taker activity they remain in the set.

Each placed order carries `expire_timestamp = clock.timestamp_ms() + order_expiration_time_ms`. If a permissionless caller arrives while neither Pyth feed has advanced past the cached timestamp (most plausibly during a Pyth-side degradation, or any window where the configured `order_expiration_time_ms` is comparable to the gap between feed updates), the executor's full ladder can sit expired in `open_orders` while the guard still treats the executor as having open orders. The skip-stale guard fires, the `cancel and settle steps` inside `refresh_quotes_inner` do not run, and the executor stops quoting until a feed advances or the admin path is invoked. Expired makers cannot transfer value to takers (when matched, DeepBook's `match_maker` generates a fill with the expired flag set, removes the maker from `open_orders`, and emits `OrderExpired` without updating `executed_quantity`), but the executor is silently absent from the book during the gap, foregoing maker rebates and leaving inventory exposed to drift.

Consider letting the skip-stale guard control only the re-placement step rather than the entire function, so the cancel and settle work runs unconditionally and clears any expired orders from `open_orders` before the freshness check decides whether to re-quote. As a lighter alternative, recording the most recent refresh's `expire_timestamp` (or the `last_refresh_ms` value proposed elsewhere in this review) on the `Executor` and bypassing the guard once that timestamp is in the past resolves the stall without restructuring the function. Either approach reduces the operational sensitivity of `order_expiration_time_ms` relative to feed cadence.

Update: Resolved in [pull request #145](#).

L-04 Orders Remain Active When Oracle Data Becomes Stale

The permissionless `refresh_quotes_permissionless` entrypoint reads Pyth prices and derives a DeepBook mid before delegating to `refresh_quotes_inner`, which only then calls `pool.cancel_all_orders`. When the oracle reads inside `market::deepbook_price` and `market::deepbook_usd_price` abort, for example when a cached Pyth price exceeds `max_price_age_secs`, the transaction reverts before reaching the cancellation step. Previously placed orders remain on DeepBook quoted against the last accepted prices, and permissionless keepers cannot retire them because every keeper-callable entrypoint depends on the same oracle read. Only the privileged `pause` function can clear them sooner.

The exposure is bounded. Each order carries `expire_timestamp = clock.timestamp_ms() + order_expiration_time_ms`, and once that timestamp is

reached DeepBook treats the maker as a no-op during matching, so the realized exposure of a stranded ladder is `min(time-to-expiry, time-to-Pyth-recovery)` rather than indefinite. Within that window, however, counterparties can still fill the ladder at off-market prices.

Consider exposing a permissionless cancel-only path that invokes `pool.cancel_all_orders` and `pool.withdraw_settled_amounts` when the regular refresh is disabled by oracle staleness, while gating it so that arbitrary callers cannot wipe the ladder during normal operation. Plausible designs include re-reading the same `PriceInfoObject`s without a `no_older_than` argument and proceeding only when a manual staleness check with a strictly looser threshold fails, tracking a `last_successful_refresh_ms` field on the `Executor` and gating on the gap between the current clock and that timestamp, or requiring the cancel path to attempt the same validity-checked oracle read as `refresh_quotes_permissionless` and proceeding only when it would abort. In every case, the threshold should be strictly looser than the freshness check used by `refresh_quotes_permissionless` and admin-tunable. As a complementary mitigation, lowering the configured `order_expiration_time_ms` directly shrinks the worst-case exposure window at the cost of more frequent re-quoting.

Update: Partially Resolved in [pull request #148](#). The OpenZeppelin development team stated:

| `order_expiration_time` is now restricted to be no more than `max_price_age`

Notes & Additional Information

N-01 Incorrect Documentation

Several documentation comments and inline comments in `executor.move` and `config.move` do not match the actual code behavior:

- The `executor_id` field of `AdminCap` is documented as "ID of the [associated market maker self](#)", but it should refer to the ID of the associated executor.
- The `deposit` function is documented as "[Emits DepositRecorded](#)", while it actually emits `Deposited`.

- The `withdraw` function is documented as "[Emits WithdrawRecorded](#)", but the correct event is `Withdrawn`.
- The `id` field of `Executor` is documented as "[Unique ID for the account object](#)", but the struct is `Executor`, not an account.
- The `config` field of `Executor` is described as "[Pool onchain configuration](#)", whereas its type is `AMMConfig`.
- The `compute_ask_price` function is documented as "[Compute ask size based on mid_price, spread](#)", but the function returns a price, not a size.
- The error messages of `EInvalidBaseSpreadBps` and `EInvalidMaxConfRatioBps` state that the value must be "at most 10000", but the corresponding assertions in `new` enforce strict inequality (`< HUNDRED_PERCENT_BPS`). Both messages should read "less than 10000" to match the actual check.
- The `new` function is documented as "[Creates a new AMM configuration object](#)", but `AMMConfig` only has the `drop` and `store` abilities (no `key`), so it is not a Sui object.

Consider updating each comment and documentation statement to accurately reflect the code, so that future readers are not misled by outdated or inaccurate information.

Update: Resolved in [pull request #141](#).

N-02 Unused Functions

The following `public(package)` functions in the `market.move` module are defined but are not called by any part of the package:

- `max_decimal_power`, which exposes the `MAX_DECIMAL_POWER` constant.
- `set_base_price_publish_time`, which overwrites the cached base publish time.
- `set_quote_price_publish_time`, which overwrites the cached quote publish time.
- `pyth_price_identifier_length`, which is referenced by the `test_helpers.move` file but not in actual production modules.

Consider removing these functions to reduce the package's surface area, or wiring them into the call sites that motivated their introduction if they are intended for future use.

Update: Resolved in [pull request #140](#).

N-03 Code Style Inconsistencies

Several minor code style inconsistencies are present in the package:

- In the `market.move` module, the `try_update_publish_time_function` is placed under the `View helpers` section, even though it is `public(package)` and mutates `Market` state. It should be relocated to the `Package Functions` section that immediately follows.
- In the `events.move` module, the `order_constructor` builds a `LimitOrder` payload, but its name does not follow the `new` / `create` convention used elsewhere for struct constructors. It should be renamed (e.g., `new_order`) so its purpose is clear at the call site.

Consider applying these adjustments to keep section organization, parameter naming, and constructor naming uniform across the codebase.

Update: Resolved in [pull request #142](#).

N-04 Arithmetic Edge Cases in Quote-Refresh Path Can Cause Aborts and Stale Orders

Several arithmetic concerns in the quote-refresh path can cause aborts when certain price or inventory configurations are reached:

- In `executor::refresh_quotes_inner`, spreads are derived from the unshifted `oracle_mid_price` via `config::base_spread` and `config::outer_spread`, while the mid used for quoting is shifted independently through `config::reservation_mid`. The derived `reservation_mid ± spread` values are then validated against `constants::min_price()` and `constants::max_price()` inside `compute_bid_price` and `compute_ask_price`, but no prior clamp guarantees those bounds hold before and after tick rounding. Reachable inventory skew and volatility configurations can therefore abort the refresh, leaving stale orders in place. Consider clamping the derived bid/ask prices into the valid range or treating out-of-range prices as "skip this order" rather than aborting, and consider bounding `volatility_multiplier_bps` so volatility-driven widening cannot produce invalid DeepBook prices.
- In `market::deepbook_price`, the oracle ratio is converted to DeepBook units by first computing `base_mantissa * constants::float_scaling_u128() / quote_mantissa` and only then applying the `10^diff` exponent adjustment.

Performing integer division before scaling causes irrecoverable truncation, the intermediate quotient can collapse to zero, in which case the subsequent `assert!(price >= constants::min_price(), ...)` aborts the refresh, or it can truncate partially and propagate a materially wrong mid into reservation pricing and spread computations. Consider applying the exponent/decimal adjustment before division so truncation happens once at the final step, ideally through a checked `mul_div` routine over `u128` with explicit rounding, and adding unit tests that cover extreme mantissa ratios and `quote_total - base_total` values.

- In `config::reservation_mid`, the inventory shift is computed as `base_spread.checked_mul(imbalance).destroy_or!(abort EPriceOverflow) / total_balance`. Since `imbalance <= total_balance` by construction, the exact quotient is always `<= base_spread`, yet the intermediate product `base_spread * imbalance` can overflow `u128` and abort with `EPriceOverflow` even when the final value is representable. This propagates into `executor::refresh_quotes_inner` and prevents re-quoting under sufficiently large prices and inventories. Consider restructuring the computation to avoid the intermediate overflow, for example by computing the ratio `imbalance / total_balance` as a fixed-point value before multiplying by `base_spread`, or by using a widened `mul_div` helper, and treating overflow defensively as a maximum shift rather than aborting to preserve quote-refresh liveness.
- In `config::base_spread`, the spread is computed as `mid_price.mul_div(base_spread_bps, HUNDRED_PERCENT_BPS)` and the constructor only constrains `base_spread_bps` to the open range `(0, 10_000)`, so the floor-divided result collapses to zero whenever `mid_price * base_spread_bps < 10_000` (e.g., `mid_price = 50` with `base_spread_bps = 100`). When `base_spread = 0` and `reservation_mid` is tick-aligned (a recurrent case since `market::deepbook_price` arithmetic uses powers of ten and the configured tick is typically a power of ten), `compute_bid_price` rounds down and `compute_ask_price` rounds up to the same tick, producing `bid_inner == ask_inner`. Because `executor::try_place_limit_orders` posts the four-order ladder in the order `[bid_outer, bid_inner, ask_inner, ask_outer]`, the third placement crosses the executor's own resting `bid_inner`. DeepBook implements the executor's `cancel_taker` self-match option inside `match_maker` as a hard `assert!(maker.balance_manager_id() != self.balance_manager_id())` rather than a silent drop, so the transaction aborts with `ESelfMatchingCancelTaker`, rolling back the cancel-and-settle and any bids that did land. Every subsequent refresh under the same configuration aborts identically until

`update_config` widens `base_spread_bps` or the mid-price scale shifts, leaving whatever was resting before the refresh stale on the book. Consider asserting `base_spread > 0` immediately after the spread computation in `refresh_quotes_inner`, so the configuration root cause surfaces as a named, operator-facing error instead of an opaque DeepBook abort, and so the executor does not post the degenerate one-tick-wide ladder produced when `base_spread = 0` and `reservation_mid` is not tick-aligned.

Consider applying these adjustments so that the quote-refresh path remains live under extreme but reachable price and inventory configurations, preventing a single arithmetic edge case from blocking re-quoting and leaving stale orders on the book.

Update: Partially Resolved in [pull request #150](#). The OpenZeppelin development team stated:

Addressed, except of sub-issue 1: clamping will introduce complication and new edge cases to be considered, for the sake of exotic price and spread values

N-05 Spelling and Grammar Errors in Comments and Documentation Affect Readability

Several inline comments and documentation strings in `executor.move` and `info.move` contain typos, grammatical errors, or ambiguous wording:

- The inline comment ["Pause step let us settle all the balances before making withdrawal"](#), repeated at [line 263](#), is grammatically incorrect and should be revised to "Pause step lets us settle all the balances before making the withdrawal."
- The inline comment ["that will force market maker resubmit orders and loose priority"](#) contains missing words and a misspelling. It should read "that would force the market maker to resubmit orders and lose priority."
- The inline comment ["Calculate bids/ask order prices around the reservation mid"](#) uses a mismatched plural. For consistency, it should read "Calculate bid/ask order prices around the reservation mid."
- The inline comment ["Try place lace 4 limit orders"](#) contains a typo. It should read "Try to place 4 limit orders."
- The documentation strings ["Returns a deepbook's trade cap ID"](#), ["Returns a deepbook's deposit cap ID"](#), and ["Returns a deepbook's withdraw cap ID"](#) include a redundant article and inconsistent capitalization. These should be updated to read "Returns the DeepBook trade cap ID", "Returns the DeepBook deposit cap ID", and "Returns the DeepBook withdraw cap ID", matching the casing used elsewhere in the file.

- The inline comments "[Will never fail, but can record invalid value \(0 or max\)](#)" and the equivalent at [line 95](#) use an ambiguous term "max". This should be clarified as "`u64::MAX`" so readers can immediately identify the intended bound.

Consider correcting each comment and documentation string to improve readability, prevent misunderstandings, and reduce ambiguity for future maintainers.

Update: Resolved in [pull request #144](#).

N-06 Live Order Ladder Quotes at Stale Parameters After `update_config`

The `update_config` function replaces the executor's `AMMConfig` and resets the cached Pyth publish timestamps, which causes the next `refresh_quotes_permissionless` call to re-price even when the oracle timestamp has not advanced. However, this function does not cancel the orders that are already resting on the DeepBook book. These orders were placed by the most recent `refresh_quotes_*` call under the previous configuration and remain active until the next refresh occurs.

As a result, the order book continues to quote at spreads, ladder sizes, and inventory skews based on a configuration that the admin has already replaced. The documentation above the function notes that `refresh_quotes_permissionless` (or the admin `refresh_quotes`) should be invoked in the same PTB as `update_config` to immediately update the quoted parameters, but the contract does not enforce this requirement. If the operator forgets to bundle the calls, separates the operations into two transactions, or if the follow-up transaction reverts, the executor may trade against external takers using outdated parameters. This could result in fills at prices or sizes that the new configuration would not have permitted.

Consider cancelling the live ladder within `update_config` so that no orders remain on the book under the outdated parameters. This would ensure that the next `refresh_quotes_*` call repopulates the book according to the new configuration.

Update: Resolved in [pull request #149](#).

N-07 Market Creation Allows Identical Pyth Feeds for Base and Quote

`market::new` binds a DeepBook pool to two Pyth feed IDs, validating the length of each (32 bytes) and the cached asset decimals, but does not assert that the two feed IDs differ. When

the same feed is passed for both sides, the `(Base/USD) / (Quote/USD)` numerator and denominator inside `deepbook_price` collapse to 1, leaving only the decimal adjustment, and the executor quotes the four-order ladder around a mid that does not reflect the real exchange rate.

Consider rejecting identical feed IDs at construction by asserting that `base_pyth_price_feed_id` differs from `quote_pyth_price_feed_id` in `market::new`, alongside the existing length and decimals checks.

Update: Resolved in [pull request #143](#).

Conclusion

The OpenZeppelin Sui market maker implements an on-chain market-making executor for the DeepBook v3 spot exchange, posting a four-order ladder around an oracle-derived mid price and exposing both a permissionless Pyth-driven refresh path and a capability-gated refresh path that accepts off-chain market data. OpenZeppelin audited the Move modules that compose the executor lifecycle, the Pyth integration, the AMM configuration validation, and the surface that drives quote generation and order placement.

During the audit, no critical- or high-severity issues were identified. Two medium-severity issues were surfaced around the interaction between the permissionless and admin refresh paths: admin-posted quotes can be overridden by the next Pyth-driven permissionless refresh, and the permissionless entry point can be invoked on every Pyth advance to force the executor to re-queue at the tail of DeepBook's FIFO priority. Four low-severity issues and several notes covered DeepBook integration assumptions on taker-side execution risk during refreshes, quoting gaps when orders expire while Pyth has not advanced, and additional opportunities to improve code clarity, consistency, and efficiency. As a general testing recommendation, boundary and edge-case tests should exercise all code paths in public-facing functions, not only in internal helpers, to further strengthen the reliability of the package. Test oracles should also remain independent of the logic under validation. When expected values are derived from the same logic that the test is meant to verify, defects in that logic can produce matching results on both sides, allowing tests to pass silently.

The OpenZeppelin development team is thanked for their responsiveness and transparency throughout the review process, including the in-depth technical explanations, clarification of architectural and design decisions, and documentation shared during the engagement, all of which facilitated the assessment.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.